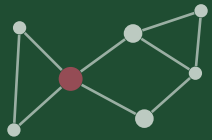
A Crash Course in Graph Databases with neo4j



Joshua D. Ingram

Department of Statistics & Data Science, Carnegie Mellon University

StatBytes, November 4, 2026

Overview

1 Overview of graphs and databases

What is a graph? Where does graph data show up? What are graph databases?

2 Database setup and Cypher queries

Setting up a neo4j database, importing data, writing Cypher queries, Python usage, and system workflows

3 Graph Data Science library

Introduction to the GDS library, graph projections, and centrality measures

4 Knowledge graphs and RAG

Knowledge graphs, retrieval-augmented generation, GraphRAG, and neo4j's AI tools

Example data: a mock regional **power grid** (140 substations, 322 lines, 25 generators, 60 loads).

PART 01

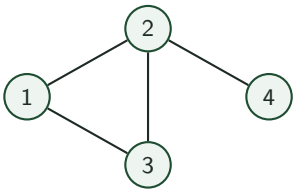
Graph data and graph databases

01

What is a graph?

A graph $G = (V, E)$ is a set of **nodes** V and a set of **edges** E between pairs of nodes. Nodes and edges can carry attributes.

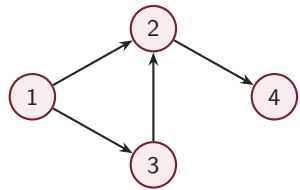
Undirected — an edge is a pair $\{i, j\}$



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

Adjacency matrix A: $A_{ij} = 1$ if i and j are joined. Symmetric. Row sums are degrees.

Directed — an edge is an ordered pair (i, j)



$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix}$$

$A_{ij} = 1$ if there is an edge from i to j. Generally not symmetric. Row sums are out-degrees, column sums in-degrees.

- Weighted graphs put a number in A_{ij} instead of 1 (distance, capacity, probability).
- Most real graphs are sparse: A is mostly zeros, so software stores *edge lists* or *adjacency lists* rather than the matrix.

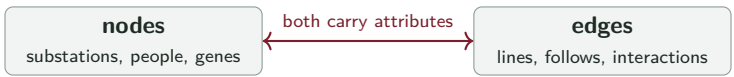
Graph data in applied settings

Social and information
people and follows, messages, co-authorship, citations, hyperlinks

Infrastructure
power grids, roads and transit, pipelines, telecom, the internet

Commerce and finance
supply chains, payments, company ownership, fraud rings, recommendations

Science
protein interactions, drug–target pairs, gene regulation, knowledge graphs



How is graph data managed at scale and in production?

Three kinds of databases

Relational

Data in **tables** with fixed columns; rows reference each other by keys; queried with SQL. Relationships are computed with joins.

PostgreSQL, MySQL, SQLite, DuckDB

id	name	kv
S012	Allegheny-12	345
S036	Laurel-01	345

Document (NoSQL)

Each record is a **JSON-like document** with its own shape; no fixed schema; fast to read and write whole records. Relationships are nested or referenced by id.

MongoDB, CouchDB, DynamoDB

```
{ "id": "S012", "kv": 345,
  "lines": ["S036", "S016"] }
```

Graph

Nodes and relationships are both stored; a relationship is a record pointing at its two ends. Queried by pattern.

neo4j, Amazon Neptune, TigerGraph, Memgraph



Graph databases

neo4j	The most widely used graph database. Property graph model, Cypher query language, open-source core, built-in algorithms library (GDS), desktop app, Docker image, cloud service (Aura).
Amazon Neptune	AWS-hosted; supports property graphs (openCypher, Gremlin) and RDF (SPARQL).
TigerGraph	Commercial, aimed at very large graphs and analytics; its own query language (GSQL).
Memgraph, ArangoDB, Kùzu	Smaller or newer: in-memory, multi-model, or embedded.

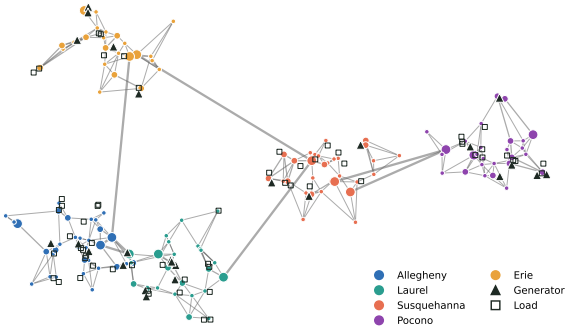
Two data models

Property graph: nodes and relationships with labels and key–value properties (neo4j, Neptune, TigerGraph).

RDF (Resource Description Framework) / triple store: everything is a (subject, predicate, object) triple, queried with SPARQL (GraphDB, Virtuoso). Common in the semantic-web world.

We'll use neo4j: it has the largest community and the most documentation.

Example data: a mock regional power grid



Nodes

- **Generator** (25): type (gas, wind, solar, hydro, nuclear), capacity in MW, cost, on/off
- **Substation** (140): voltage level (345 / 138 / 69 kV), region, on/off, location
- **Load** (60): residential, industrial, or hospital; demand in MW

Edges

- **LINE** (322): substation to substation; capacity, length, annual fault probability
- **FEEDS** (25): generator to substation
- **SUPPLIES** (60): substation to load

Synthetic power grid built to mimic the Mid-Atlantic power grid, with five regions.

The grid in a relational database and in neo4j

Relational: two tables, joined by keys

substation

id	name	kv
S012	Allegheny-12	345
S036	Laurel-01	345
S044	Laurel-09	345

line

from	to	fault_prob
S012	S036	0.07
S036	S044	0.03

```
-- substations two lines away from S012
SELECT b.to
FROM line a JOIN line b ON a.to = b.from
WHERE a.from = 'S012';
```

Each extra hop is another join over the whole `line` table.

Graph: substations are nodes, lines are relationships




```
// substations two lines away from S012
MATCH (a:Substation {id: 'S012'})
      -[:LINE]-()-[:LINE]-(b)
RETURN b
```

The query is the picture: nodes in parentheses, relationships in brackets. Each hop follows stored pointers from the current node.

A first Cypher query

Look up one row

```
SELECT *
FROM substation
WHERE name = 'Allegheny-01';
```

```
MATCH (s:Substation {name: 'Allegheny-01'})
RETURN s
```

- `MATCH` finds things, `RETURN` gives them back
- `(s:Substation {...})` is one node: variable `s`, label `Substation`, a property filter

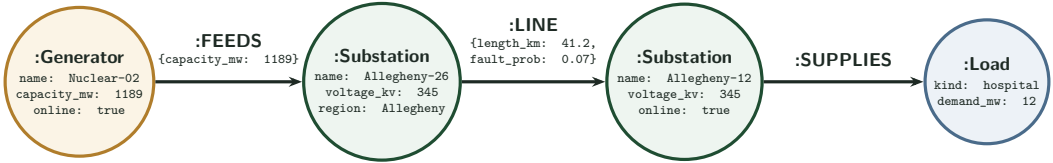
Then one step further

```
SELECT t.*
FROM substation s
JOIN line l       ON l.from = s.id
JOIN substation t ON t.id = l.to
WHERE s.name = 'Allegheny-01';
```

```
MATCH (s:Substation {name: 'Allegheny-01'})-[:
    LINE]-(t)
RETURN t
```

- `-[:LINE]-` is a relationship of type `LINE`, either direction (the SQL needs a union for that)
- the pattern reads the way the graph is drawn

The property graph model, with the grid



Nodes

One or more **labels** (`:Substation`) and **properties**: numbers, strings, booleans, lists, dates, points.

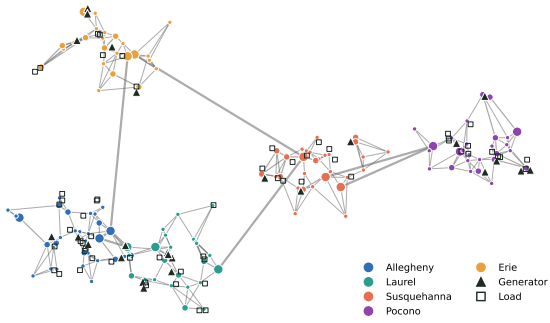
Relationships

Exactly one **type** (`:LINE`), a **direction**, and their own **properties**. Always join two existing nodes.

Schema

Optional. No table definitions up front; constraints and indexes are added where needed. Mixed node types are normal.

The grid as a Cypher pattern




```
(:Generator {type, capacity_mw,
             marginal_cost, online})
    -[:FEEDS {capacity_mw}]->
(:Substation {voltage_kv, region,
              online, lat, lon})
    -[:LINE {capacity_mw, length_km,
             fault_prob}]-
(:Substation)
    -[:SUPPLIES]->
(:Load {kind, demand_mw})
```

The whole data model fits in one pattern. A query is a piece of this pattern with some values filled in, for example "a nuclear generator, its substation, and the hospitals within three lines".

Direction is stored (`FEEDS` goes generator → substation) and can be ignored when querying (`-[:LINE]-`).

When to use a graph database

Good fit

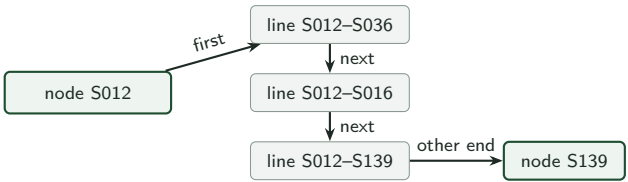
- questions about **paths and neighborhoods**: within k hops, is there a route, what connects A to B
- **many entity types** with relationships among them, and a schema that keeps changing
- the graph is **larger than memory** or used by several people and services at once
- queries, indexes, transactions, and graph algorithms on one copy of the data

Poor fit

- **column aggregates** over millions of rows: relational and columnar databases are faster
- a graph that **fits in memory**, analyzed once: `networkx`, `igraph`, or `graph-tool` are simpler and faster
- tabular analyses with one or two joins
- work that needs the **adjacency matrix** itself

A graph database is a **storage and query** decision.

How neo4j stores a graph



Every node record holds a pointer to its first relationship; relationships are chained in a list and each one points at both of its end nodes. Moving to a neighbor means following a pointer.

What this buys

- a hop costs the same whether the graph has a thousand nodes or a billion
- a k-hop query touches only the part of the graph it walks through

What it does not

- finding the *starting* node ("the substation named X") still needs an index, which you create with a constraint
- writes are a little slower, because the lists have to be maintained

PART 02

Setup and Cypher

02

Setting up your own database

Docker (used today)

```
cd demo && docker compose up -d
# then open http://localhost:7474
```

Port 7474 is the Browser, 7687 the driver port; `./data` is mounted at `/import`.

```
services:
  neo4j:
    image: neo4j:5
    ports: ["7474:7474", "7687:7687"]
    environment:
      NEO4J_AUTH: neo4j/statbytes2026
      NEO4J_PLUGINS: '["graph-data-science"]'
    volumes: ["./data:/import"]
```

Neo4j Desktop (no Docker)

1. Download from `neo4j.com/download`
2. Add → Local DBMS, choose a password
3. Plugins tab → install Graph Data Science
4. Copy the CSVs into the DBMS's `import` folder
5. Start, open Browser

Aura (hosted)

Free tier, nothing to install. No GDS on the free tier (AuraDS is the paid option). Fine for the Cypher part of this talk.

Loading data

```
// a uniqueness constraint also creates an index
CREATE CONSTRAINT sub_id IF NOT EXISTS
  FOR (s:Substation) REQUIRE s.id IS UNIQUE;

// nodes: one CSV row -> one node
LOAD CSV WITH HEADERS
  FROM 'file:///substations.csv' AS row
CREATE (:Substation {
  id: row.id, name: row.name,
  voltage_kv: toInteger(row.voltage_kv),
  region: row.region,
  online: toBoolean(row.online)
});
```

```
// relationships: find both ends, connect them
LOAD CSV WITH HEADERS
  FROM 'file:///lines.csv' AS row
MATCH (a:Substation {id: row.from_id}),
      (b:Substation {id: row.to_id})
CREATE (a)-[:LINE {
  capacity_mw: toInteger(row.capacity_mw),
  length_km:   toFloat(row.length_km),
  fault_prob:  toFloat(row.fault_prob)
}]->(b);
```

- CSV values arrive as strings; cast them
- the `MATCH` on `id` uses the index from the constraint
- for very large files: `neo4j-admin database import`, or batched writes from a driver

The full script is `cypher/01_load.cypher`: three constraints, six `LOAD CSV` blocks, one count at the end.

Cypher queries

```
MATCH (g:Generator {type: 'nuclear'})-[:FEEDS]->(s:Substation)-[:LINE*1..3]-(t:Substation)
WHERE t.online AND t.voltage_kv >= 138
RETURN g.name, t.region, count(DISTINCT t) AS reachable
ORDER BY reachable DESC
```

`(g:Generator {type: 'nuclear'})`	a node: variable, label, property filter
`-[:FEEDS]->`	a relationship with a type and a direction
`-[:LINE*1..3]-`	one to three hops, either direction
`WHERE ... RETURN ... ORDER BY`	filter, project, aggregate; the non-aggregated columns are the groups

Reading

`MATCH` a pattern, `WHERE` to filter, `RETURN` to project. `WITH` passes rows to another `MATCH`.

Writing

`CREATE`, `MERGE` (create if missing), `SET`, `DELETE`. The same pattern syntax.

Paths

`p = (a)-[*]-(b)` binds a path; `nodes(p)`, `length(p)`, `shortestPath()`. Bound variable-length patterns.

Live: eight queries

	Query	What it shows
Q1	counts by label and type	the shape of the data: `labels(n)`, `type(r)`
Q2	look up Allegheny-01	a lookup; the same thing as `SELECT ...WHERE`
Q3	its neighbors, as a path	`RETURN p` and the Browser draws the result
Q4	nuclear plant, substation, one line out	three node types in one pattern
Q5	generation vs demand by region	`WITH` chains two matches; aggregates
Q6	hospitals within 3 lines of a nuclear plant	`[:LINE*1..3]`, `DISTINCT`
Q7	`shortestPath()` Allegheny-01 to Pocono-01	path variables, `nodes(p)`
Q8	set an `inspected` date	a write, with the same syntax

Browser tips

`Ctrl+Enter` runs. The result panel switches between graph, table, and text. `:schema` lists constraints. `EXPLAIN` or `PROFILE` in front of a query shows the plan.

Using Python with neo4j

```
# pip install neo4j graphdatascience
from graphdatascience import GraphDataScience

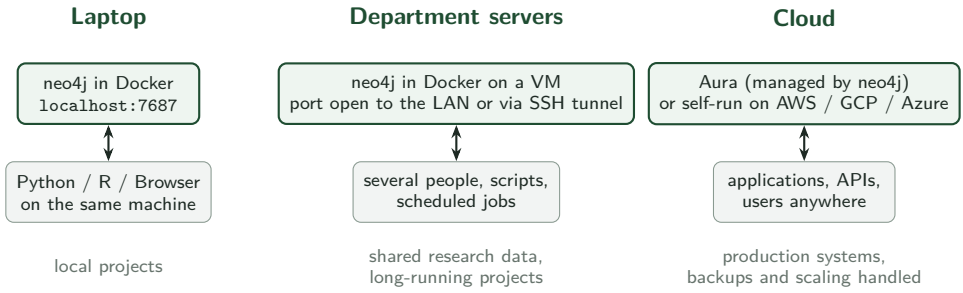
URI, AUTH = "bolt://localhost:7687", ("neo4j", "statbytes2026")
gds = GraphDataScience(URI, auth=AUTH)
q = gds.run_cypher   # returns a DataFrame

query = """MATCH (s:Substation)-[:SUPPLIES]->(l:Load)
           WHERE s.region = $region
           RETURN l.kind, sum(l.demand_mw) AS mw"""
df = q(query, {"region": "Pocono"})
```

```
# the plain driver, without the GDS client
from neo4j import GraphDatabase
drv = GraphDatabase.driver(URI, auth=AUTH)
df = drv.execute_query(query, region="Pocono",
       result_transformer_=lambda r: r.to_df())
```

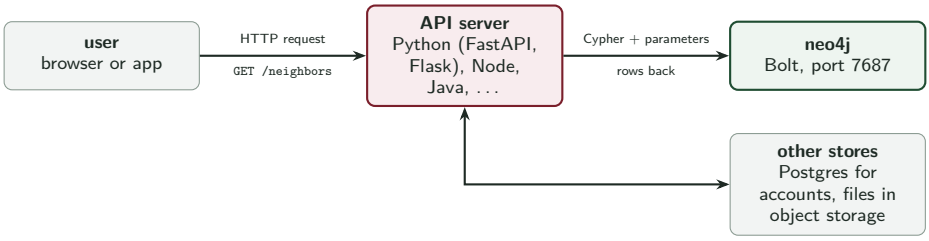
- the connection is over **Bolt** (the protocol Python and neo4j use to communicate); the same code works for a local container, a server, or Aura
- **parameters** (`$region`) keep the query text fixed
- every result is a **DataFrame**
- send the question to the database and get a summary back; for local algorithms, pull an edge list once

In practice: where the database lives



- The code is the same in all three: a Bolt URL, a user, a password.

In practice: a graph database behind a web application



The usual pattern

- the API server holds a pool of driver connections and a read-only database user; users never reach the database directly
- each endpoint is one Cypher query with parameters; results go back as JSON
- GDS jobs run on a schedule and write results back; endpoints only read

Typical uses

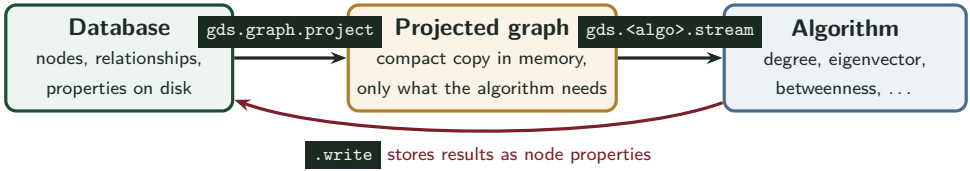
- recommendations: people who bought this also bought
- fraud: rings of accounts sharing devices and addresses
- network operations and digital twins: outages, dependencies, what-if

PART 03

Graph Data Science

03

The Graph Data Science library




```
CALL gds.graph.project('grid', 'Substation',
  { LINE: { orientation: 'UNDIRECTED' } });

CALL gds.degree.stream('grid')
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name, score
ORDER BY score DESC LIMIT 10;
```

- a plugin that runs inside the database; called from Cypher with `CALL gds....`
- most algorithms have four modes: `stream` (rows back), `stats`, `mutate` (update the projection), `write` (update the database)

What is in the library

Centrality

degree, eigenvector, PageRank, betweenness, closeness

which nodes are important?

Community and components

connected components, Louvain, Leiden, label propagation, triangle counts

what are the groups?

Paths

shortest paths (Dijkstra, A*), k shortest paths, spanning trees, random walks

how do I get from A to B?

Similarity, embeddings, ML

node similarity, k-nearest neighbors, FastRP and node2vec embeddings, link prediction pipelines

which nodes are alike?

GDS documentation provides a description and the requirements of each algorithm.

Graph projections: what and why

```
// native projection: labels and relationship types
CALL gds.graph.project(
  'grid',                    // a name
  'Substation',              // which nodes
  { LINE: { orientation: 'UNDIRECTED' } }
)
YIELD nodeCount, relationshipCount;
// -> 140 nodes, 644 relationships

// Cypher projection: any pattern you can write
MATCH (s:Substation)-[:LINE]-(t:Substation)
WHERE s.online AND t.online
RETURN gds.graph.project('grid_online', s, t);
```

What it is. A copy of part of the graph in a compact in-memory format, with a name. Algorithms run on the projection, never on the stored data.

Why.

- algorithms need the whole graph in a form built for fast iteration; the storage format is built for lookups and updates
- you choose the subgraph: only substations and lines, only online nodes, only one region
- you choose the direction: `UNDIRECTED` for a grid, `NATURAL` for a citation network
- one projection serves many algorithms, then is dropped

Undirected lines are stored both ways, so 322 lines become 644 relationships.

Degree centrality

The number of edges at a node. With the adjacency matrix A,

$$d_i = \sum_j A_{ij}.$$

Directed graphs have an in-degree (column sum) and an out-degree (row sum). With weights, the sum of the weights.

What it says. How many direct connections a node has; for the grid, how many lines meet at a substation.

Limits. Purely local. Eight lines to dead ends score the same as eight lines to other hubs.

```
CALL gds.degree.stream('grid')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS s, score
RETURN s.name, s.region, toInteger(score) AS degree
ORDER BY score DESC LIMIT 10;
```

```
df = q("CALL gds.degree.stream('grid') YIELD nodeId, score "
       "RETURN gds.util.asNode(nodeId).name AS name, score")
```

Options: `orientation` and `relationshipWeightProperty` (weighted degree).

Eigenvector centrality

A node scores high if its neighbors score high. Each score is proportional to the sum of its neighbors' scores:

$$x_i = \frac{1}{\lambda} \sum_j A_{ij}\, x_j \quad \Longleftrightarrow \quad A\,x = \lambda\,x.$$

x is the leading eigenvector of A; for a connected graph it is unique and positive.

What it says. Connections to well-connected nodes count for more than raw degree. PageRank is a damped variant for directed graphs.

Limits. Dominated by the densest cluster; with weakly joined regions, one region can take all the mass.

```
CALL gds.eigenvector.stream('grid', { maxIterations:
     200 })
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS s, score
RETURN s.name, s.region, round(score, 3) AS
     eigenvector
ORDER BY score DESC LIMIT 10;
```

```
df = q("CALL gds.eigenvector.stream('grid') YIELD nodeId, score "
       "RETURN gds.util.asNode(nodeId).name AS name, score")
```

Options: `maxIterations`, `tolerance`, `scaler`. By default the returned vector has unit length (L2 norm 1).

Betweenness centrality

The share of shortest paths that pass through a node. With σ_{st} the number of shortest paths from s to t and $\sigma_{st}(v)$ the number of those that go through v,

$$b_v = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}.$$

Computed with Brandes' algorithm: one breadth-first search per source node, $O(nm)$ in total.

What it says. Which nodes sit on the routes between other nodes; for the grid, which substations carry traffic between regions.

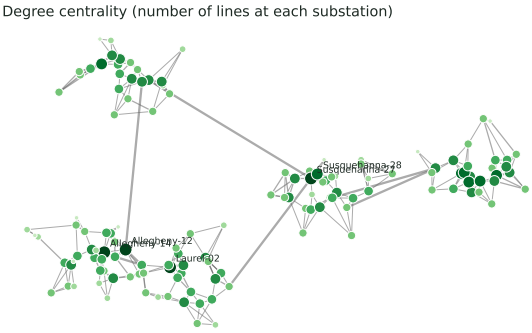
Limits. Assumes flow follows shortest paths. Cost grows fast with size; GDS has a sampled version (`samplingSize`).

```
CALL gds.betweenness.stream('grid')
YIELD nodeId, score
WITH gds.util.asNode(nodeId) AS s, score
RETURN s.name, s.region, round(score) AS betweenness
ORDER BY score DESC LIMIT 10;
```

```
df = q("CALL gds.betweenness.stream('grid') YIELD nodeId, score "
       "RETURN gds.util.asNode(nodeId).name AS name, score")
```

GDS returns raw path counts. Divide by $(n-1)(n-2)/2$ for the usual $[0, 1]$ scale on an undirected graph.

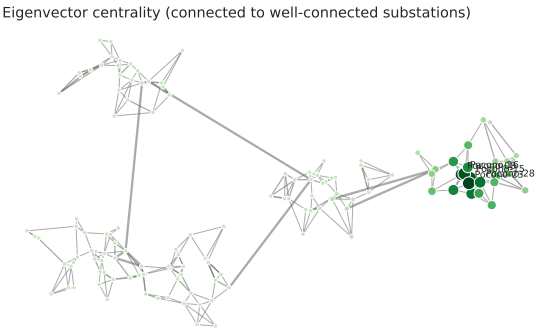
In practice: degree



substation	region	kV	degree
Allegheny-12	Allegheny	345	8
Allegheny-14	Allegheny	69	8
Laurel-02	Laurel	69	8
Susquehanna-27	Susquehanna	345	8
Pocono-08	Pocono	345	7
Pocono-11	Pocono	138	7

- projection: 140 nodes, 644 relationships
- the top of the list mixes 345 kV hubs with 69 kV substations that happen to have many short local lines
- degree alone does not separate "important for the grid" from "many small connections"

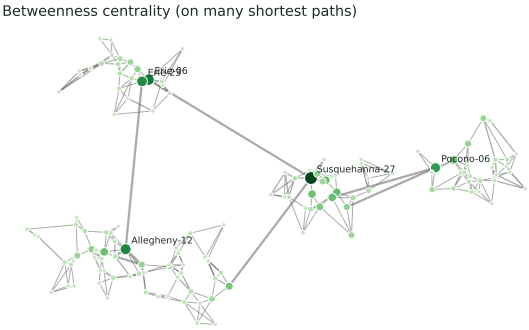
In practice: eigenvector



substation	region	kV	score
Pocono-23	Pocono	69	0.353
Pocono-16	Pocono	69	0.331
Pocono-08	Pocono	345	0.329
Pocono-15	Pocono	69	0.322
Pocono-28	Pocono	69	0.283
Pocono-05	Pocono	69	0.278

- the whole top ten is in Pocono, the most densely connected region
- this is the limit from the previous slide: the leading eigenvector concentrates on the densest cluster
- useful within one region; misleading as a grid-wide ranking

In practice: betweenness

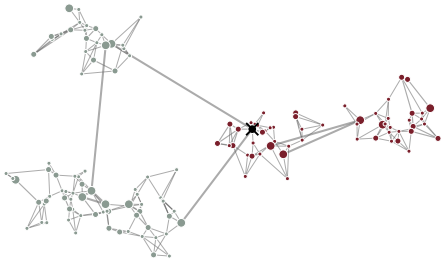


substation	region	kV	score
Susquehanna-27	Susquehanna	345	5171
Erie-06	Erie	345	3871
Allegheny-12	Allegheny	345	3600
Erie-23	Erie	345	3431
Pocono-06	Pocono	345	3038
Susquehanna-03	Susquehanna	69	1939

- the top five are all 345 kV substations at the ends of the inter-ties between regions
- this is the ranking that matches "which substation would hurt most if it failed"
- Susquehanna-27 is on more than half of all shortest paths between pairs of substations (5171 of 9591)

What if a node broke? Connected components

Susquehanna-27 offline: 2 components; 57 substations cut off (red)



subs	regions	gen	demand	hosp.
82	Allegheny, Erie, Laurel	5474	2807	4
57	Pocono, Susquehanna	1488	1436	2

```
MATCH (s:Substation {name: 'Susquehanna-27'})
SET s.online = false;
MATCH (s:Substation)-[:LINE]-(t:Substation)
WHERE s.online AND t.online
RETURN gds.graph.project('grid_online', s, t);
CALL gds.wcc.stream('grid_online')
YIELD nodeId, componentId ...
```

Weakly connected components: the groups of nodes that can reach each other by some path, ignoring direction. Each group is an electrical island.

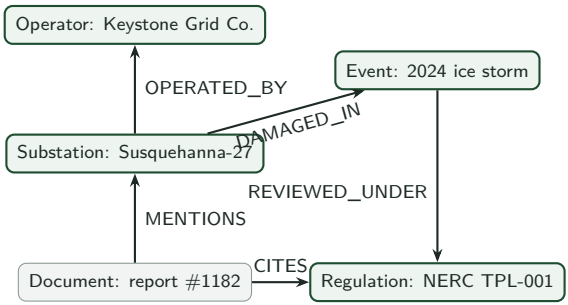
What we learn. One substation splits the grid in two. The eastern island keeps 1488 MW of generation for 1436 MW of demand and two hospitals, which leaves little margin.

PART 04

Knowledge graphs and RAG

04

Knowledge graphs



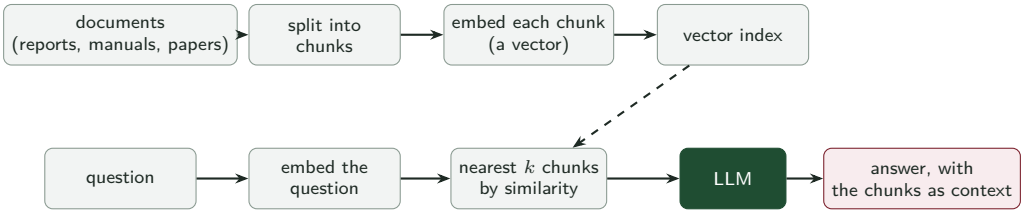
A **knowledge graph** is a property graph whose nodes are real-world entities and whose relationships are facts about them. Three things usually come with it:

- **schema (ontology)**: the allowed entity types and relationship types, sometimes with a hierarchy
- **provenance**: which document or source asserted each fact, and when
- **identity resolution**: "Susquehanna-27", "SUSQ27", and "the Milton substation" are one node

Examples: Wikidata, Google's Knowledge Graph, Hetionet, digital twins of grids and factories.

Language models read text well and recall exact facts unreliably. A knowledge graph keeps the facts; a language model can help build one from text.

Retrieval-augmented generation (RAG)



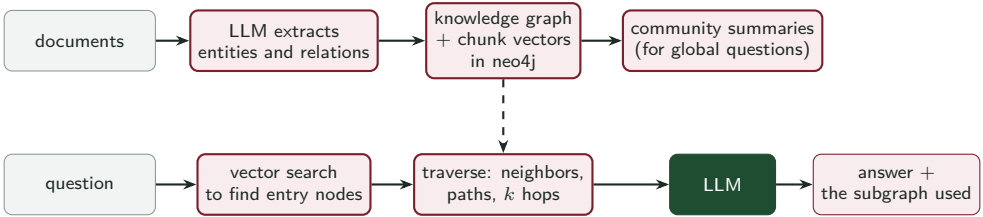
Why it is used

- answers are grounded in your documents, with citations
- no retraining: add a document and it is searchable
- good for "find the passage that answers this"

Where it falls short

- **multi-hop**: "which hospitals depend on substations run by the operator fined in 2024?" is in no single chunk
- **global**: "recurring causes across all reports?" needs aggregation, not similarity
- similar wording is not the same as a real relationship

GraphRAG



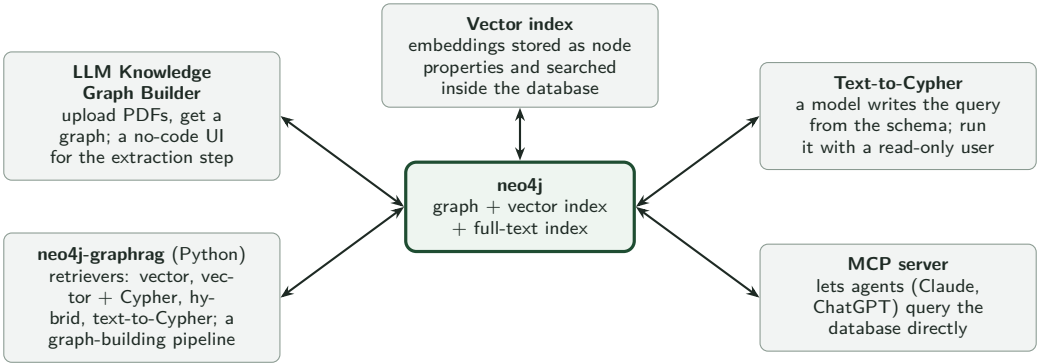
Workflow

- build: a model reads each document and proposes entities and relationships; they are merged into a graph with provenance
- retrieve: vector search finds a starting point; traversal collects the connected context
- answer: the model gets the subgraph and the source chunks

Advantages

- multi-hop questions become path queries
- global questions use precomputed community summaries (Microsoft's GraphRAG, 2024, uses Leiden)
- the retrieved subgraph can be shown, so answers can be checked

neo4j's AI tools



- With the vector index, chunk embeddings and the knowledge graph are in one database: find by similarity, then expand by relationship, in one query.
- `neo4j-graphrag` is the official Python package for building and querying; text-to-Cypher and the MCP server are how a model asks the database questions on its own.

GraphRAG: when it helps and when it does not

Use it when

- questions are relational: who is connected to what, through which steps
- the corpus is large and answers draw on many documents at once
- answers must be auditable: the subgraph is the evidence
- the same entities also need ordinary queries, reports, or dashboards

Plain vector RAG is enough when

- a few hundred documents and single-passage questions
- the extra cost of extraction (tokens, time, curation) is not justified

Watch out for

- extraction quality: the same relation under five names, entities that split or merge; a fixed schema and identity resolution are required
- generated Cypher that is wrong or expensive; use a read-only user and query timeouts

Takeaways

1 Graphs as data

A graph database stores relationships next to nodes. Traversals cost what they touch. Use one when questions follow paths or the graph outgrows memory.

2 Cypher

Queries are drawn: `(a)-[:LINE*1..3]-(b)`. The same patterns read and write. Docker gives you a database in two minutes.

3 GDS

Project, run, write back. Degree is local, eigenvector follows density, betweenness finds the bridges. Components show what an outage does.

4 Knowledge graphs

Vector search to land, graph to expand, subgraph to check. The hard part moved to building the graph well.

Questions?

Docs: neo4j.com/docs/cypher-manual · neo4j.com/docs/graph-data-science · neo4j.com/docs/neo4j-graphrag-python

PART A

Appendix



Cypher cheat sheet

```
// patterns
(n)  (n:Label)  (n:Label {k: v})  (a)-[r:TYPE]->(b)
(a)-[:T*2..4]-(b)        // 2 to 4 hops, any
     direction
p = (a)-[*]-(b)          // bind a path; nodes(p),
     length(p)
shortestPath((a)-[:T*]-(b))

// reading
MATCH ... WHERE ... RETURN ... ORDER BY ... LIMIT
     ...
OPTIONAL MATCH           // left join
WITH x, count(*) AS n    // pipe + aggregate
UNWIND list AS item      // list -> rows
EXISTS { (a)-[:T]->() }  // subquery predicate
```

```
// writing
CREATE (n:Label {k: v})
MERGE (n:Label {id: 1}) ON CREATE SET ... ON MATCH
     SET ...
SET n.k = v   REMOVE n.k   DELETE r   DETACH DELETE
      n

// schema
CREATE CONSTRAINT ... FOR (n:L) REQUIRE n.id IS
     UNIQUE
CREATE INDEX ... FOR (n:L) ON (n.k)
CREATE VECTOR INDEX ... FOR (n:L) ON (n.emb)
     OPTIONS {...}

// functions
labels(n) type(r) keys(n) properties(n) elementId(n
     )
toInteger toFloat toBoolean toString date() point()
size(list) [x IN list | f(x)]  collect() count(
     DISTINCT x)
```

Fallback: Cypher demo results

Q1 counts

Substation	140
Load	60
Generator	25
LINE	322
SUPPLIES	60
FEEDS	25

Q2 Allegheny-01: 69 kV, Allegheny, online

Q7 shortestPath Allegheny-01 to Pocono-01: 11 hops

Q5 generation vs demand (MW)

region	generation	demand
Allegheny	1846	1450
Erie	715	728
Laurel	2913	629
Pocono	750	797
Susquehanna	738	639

Q6 hospitals within 3 lines of a nuclear plant

plant	hospital	via
Nuclear-02	Hospital-01	Allegheny-24
Nuclear-02	Hospital-02	Allegheny-22

Q3 / Q4 graph views: Allegheny-01 with its neighbors; two small stars around Nuclear-01 (Laurel) and Nuclear-02 (Allegheny).

Q8 Allegheny-01, `inspected` = today's date.

Fallback: GDS demo results

G0 projection: 140 nodes, 644 relationships

G1 degree

Allegheny-12	345	8
Allegheny-14	69	8
Laurel-02	69	8
Susquehanna-27	345	8
Pocono-08	345	7
Pocono-11	138	7
Pocono-16	69	7
Pocono-23	69	7

G2 eigenvector (unit-length vector)

Pocono-23	69	0.353
Pocono-16	69	0.331
Pocono-08	345	0.329
Pocono-15	69	0.322
Pocono-28	69	0.283
Pocono-05	69	0.278
Pocono-01	138	0.253
Pocono-25	69	0.240

G3 betweenness (raw counts; divide by 9591 to normalize)

Susquehanna-27	345	5171
Erie-06	345	3871
Allegheny-12	345	3600
Erie-23	345	3431
Pocono-06	345	3038

G4 components with Susquehanna-27 offline (139 nodes projected)

subs	regions	gen	dem	h
82	Alleg., Erie, Laurel	5474	2807	
57	Pocono, Susq.	1488	1436	

G5 `betweenness.write`: 140 properties written; top five as in G3.

Figures for G1–G4 are on slides 30–33.